

ProjectVL • Unity 移植交付包

交接说明 + 主契约 + 黄金回放规格

交付日期：2026-07-27 | 面向：Unity 开发者（含其 AI Agent）

交给 Unity 开发者 · 交接说明与文件清单

日期：2026-07-27 | 交付人：项目负责人 | 面向：Unity 开发者（含其 AI Agent）

先看这一页就够开工。它告诉你三件事：**接收哪些东西、按什么顺序做、怎么带着 AI Agent 做**。更细的规则在《Unity移植_纵向切片交付说明.md》里，本页会指你去看对应章节。

一句话背景

有一个用 TypeScript 写的网页原型（H5），它是这个游戏规则的**参考实现**。你的任务**不是把网页翻译成 Unity**，而是：先在 Unity 里打通"一条最小的完整链路"，用一批固定种子的对照样本证明"你的规则算出来的结果和参考实现一致"，然后再一类一类往外扩。参考实现只当"标准答案"和调试基准，它的前端代码你一行都不用抄。

一、你会收到的东西（交接清单）

整个仓库都给你，但真正要盯的是下面这些。分三类。

A. 先读的文档（按顺序读）

顺序	文件	作用
1	docs/Unity移植_纵向切片交付说明.md	你的主契约 。开工就绪定义、一期范围、每帧执行顺序、对照协议、三条硬约束都在这。本页是它的导读
2	docs/黄金回放_fixture规格.md	对照样本怎么读、rng 算法与自检向量、逐项容差、出问题按什么顺序排查
3	docs/效果原子参数契约_落地记录.md	33 个效果原子的参数/默认值来源核对结果，实现解释器时对照
4	docs/接下来任务计划_v1.md § 1	模块三分类（哪些稳定可移植、哪些实验先别碰、哪些是延后的预留结构位）
5	docs/移植准入与配置契约固化_决策结论_v1.md	背景决策（为什么 JSON 是唯一权威源、为什么先切片）。当参考，不必逐字读

B. 要当"标准答案"的规则源文件（只读，用来照着实现）

文件	你要从中拿到什么
src/core/updateGame.ts	每帧的执行顺序 （最关键，顺序错了两端结果就分叉）。主契约 § 三逐条抄了它
src/core/createInitialState.ts	初始状态有哪些字段、初值是多少

文件	你要从中拿到什么
src/core/effects/atomContract.ts	效果原子的 权威参数表 。默认值一律以这张表为准
src/core/effects/defs.ts	效果/卡牌的类型定义，用来生成你的 C# 数据结构
src/core/rng.ts	随机数算法（mulberry32）。 Unity 必须实现同一个算法 ，否则对照必失败
src/config/loader.ts	配置怎么深合并（base + variant 覆盖）的语义
src/config/affixSinks.ts	词条"落点"契约，派生属性重算时对照
src/core/replay/record.ts	参考实现是怎么跑录制/回放的，你可照它的流程搭 Unity 侧对照 harness

C. 直接读取、不要改的数据文件

文件/目录	说明
src/config/base/*.json（14 个域）	全部数值配置。 JSON 是唯一权威源，Unity 只读不写
src/config/variants/dev-short.json	一个覆盖示例，用来验证你的深合并实现
src/data/texts.json	文案表，导入 Unity Localization
tests/golden/*.spec.json + *.summary.json	5 组对照样本 ：.spec 是输入，.summary 是期望输出。你的验收就是重放 .spec 要算出和 .summary 一致的结果

记住一条铁律：**所有数值来自 JSON**。你可以把 JSON 导进 Unity 当 ScriptableObject 缓存，但那只是缓存，不是第二套真相；发现数值有问题，反馈回来改 JSON，别在 Unity 里私自改。

二、你接下来一步步做什么

按这个顺序，每步做完再做下一步。

第 0 步 · 环境与自检（半天） 把仓库拉下来，`npm install` 后跑一次 `npm run test`，看参考实现全绿——这是你的"标准答案机器"，之后随时可查。然后在 Unity 里做第一件小事：实现 `rng.ts` 里的 `mulberry32`，用 `seed=42` 跑前 8 个随机数，和规格书 § 2 的自检向量对上。**这一步过不了，后面全白做**，所以先做它。

第 1 步 · 读三样东西再动手 读主契约 § 三（每帧执行顺序）、§ 六（对照协议）、规格书全篇。别急着写代码，先在脑子里（或纸上）把"一帧里发生什么、按什么顺序"过一遍。

第 2 步 · 搭规则层骨架 在 Unity 里建一个**不依赖引擎**的规则层：它只接收 `state + config + dt + rng`，返回新状态和一连串事件（`GameEvent`）。表现、输入、UI 都在这层之外，靠事件驱动。这层里**不许出现 `using UnityEngine`**。这是能不能对照成功的结构前提。

第 3 步 · 打通一条最小链路（一期的核心） 按主契约 § 二的 12 步，实现"读配置 → 建初始状态 → 固定步推进 → 出一种敌人 → 炮台自动开火结算 → 一次掉落拾取 → 一次合成 → 一张装备态卡 → 一张消耗态卡 →

一个状态效果 → 把事件抛给表现层 → 和对照样本比对”。只做这一条链路用到的东西，不做别的（不做全部卡牌、全部怪、神池、进化、Bounty——那些是后面的事）。

第 4 步 · 跑黄金对照 用你的规则层重放 `tests/golden/` 里 5 个 `.spec.json`，把结果和对应 `.summary.json` 比。判据：事件类型序列、掉落序列、通关/失败结果、波次推进、随机数抽取次数必须一致；纯数值按规格书 § 5 的容差（不要求逐位相同，要求统计一致）。**第 1 个样本 01-slice-combat 无输入无决策，是你最容易先过的，从它开始。**

第 5 步 · 对不上就查顺序 任何一项对不上，先别怀疑数值——先查执行顺序（主契约 § 3.1）和判定次序（§ 3.2），九成分叉出在这两处。规格书 § 6 给了固定的排查顺序，照着走。

第 6 步 · 一期验收通过后再扩张 5 个样本全过 = 一期完成。之后按这个顺序一类一类加：弹道 → 控制 → 领域 → 防御 → 经济 → 神池/遗物/进化。每加一类，补几个对照样本，再验一次。

三、怎么和你的 AI Agent 配合（重要）

你会用 AI Agent 写代码。用得好能快很多，用不好会算出一堆“看起来对、其实和参考实现分叉”的代码。下面几条是这个项目里被验证有效的做法：

1. 每个 prompt 先让 Agent 读契约，再动手。 不要直接说“实现炮台开火”。要说“先读 `docs/Unity移植_纵向切片交付说明.md` § 3.1 和 `src/core/updateGame.ts`，然后按里面的执行顺序实现这一帧的推进”。把契约文档当 Agent 的只读事实源，每个任务都指定它先读哪几节。这个项目的规则严重依赖“顺序”，Agent 不读顺序就会自己瞎排。

2. 按 12 步切片喂，别让它一次啃整个游戏。 一个 prompt 做一步（或半步），做完立刻验。一次性让 Agent“把游戏做出来”必然失控。参考实现这边就是这么一个个小任务推进的。

3. 让黄金 fixture 当 Agent 的自动验收，别自己当人肉测试机。 每个 prompt 结尾都加一句：“写完后重放 `tests/golden/01-slice-combat.spec.json`，和 `.summary.json` 比对，不一致就自己按规格书 § 6 排查并修，直到通过再向我汇报。”让 Agent 自己拿标准答案对，自己修，你只在它卡住时介入。这是这套黄金基线存在的全部意义。

4. 把三条铁律写进每个 prompt 的约束里：

- 不许为某一张卡写 `if`。卡是数据（JSON）+ 通用解释器；Agent 一旦开始 `if (card.id == ...)` 就是走错路，立刻拦住。
- `core/规则层` 不许 `import` 引擎。一旦 Agent 在规则层写 `UnityEngine`，结构就烂了。
- 配置只读。Agent 不许为了让结果对上而去改 JSON 数值；数值有疑问是反馈给我，不是它自己改。

5. 让 Agent 每完成一类就 commit + 写一份落地记录。 和参考实现这边的做法一样：每个阶段产出一份简短的“落地记录”文档（做了什么、和标准答案的差异是多少、遗留什么）。这样你（和我）能随时审，出问题也能回溯。

6. 分叉时，让 Agent 按固定套路排查，而不是瞎试。 两端结果对不上时，给 Agent 的指令是：“按规格书 § 6 的排查顺序，先核对执行顺序 § 3.1，再核对判定次序 § 3.2，逐项定位分叉的第一帧。”不要让它随机改数值碰运气。

四、你完成一期的标准

一句话：**5 个黄金样本全部对照通过，且你的规则层不依赖 Unity 引擎、配置全部只读自 JSON。**达到这个，就回来找项目负责人，我们再一起定"按类别扩张"的下一批切片。

有任何契约层面的疑问（顺序、判定、容差、数值），回来问，别自己在 Unity 侧改规则或改数值——那会让两端悄悄分叉，是这个项目最贵的错误。

Unity 移植 · 纵向切片交付说明

日期：2026-07-26 | 交付对象：Unity 开发者 | 上游：docs/接下来任务计划_v1.md

本文件是给 Unity 开发者的开工契约。**目标不是把整个网页游戏翻译成 Unity，而是先打通一条端到端链路验证接口，再按原子类别扩张。** H5 原型是"可执行的规则参考实现"与调试基准，不是要照搬的前端。

一、开工就绪定义 (Definition of Ready)

以下全部满足即可开工（不必等 H5 侧的契约固化代码全部完成）：

- ☑模块三分类清单已定（见 docs/接下来任务计划_v1.md §1）。
- ☑核心更新契约已定（本文件 §三）。
- ☑配置权威源 = JSON，Unity **只读不反向写**（见 §五）。
- ☑第一版效果原子参数契约已定：src/core/effects/atomContract.ts（33 原子的参数/默认值/触发器/形态支持），核对记录见 docs/效果原子参数契约_落地记录.md。
- ☑一组固定 seed 黄金 fixture 已产出：tests/golden/，字段定义与比对口径见 docs/黄金回放_fixture规格.md。

二、Unity 一期只做纵向切片（范围边界）

做：一条端到端链路 + 一次跨引擎对照。**不做：**全部卡牌、全部遗物、全部文案、完整调参编辑器、全部 VFX、全部怪物机制、神池/进化/Bounty。

链路（12 步，必须按 H5 的语义实现）：

- 读取同一套 JSON 配置（config/base/*.json）。
- 构建基础 GameState（对齐 core/createInitialState.ts 的字段与初值）。
- 固定时间步推进（dt 语义见 §三）。
- 出现一种敌人（enemies.json 里最基础的一型）。
- 炮台自动索敌 + 开火 + 伤害结算。
- 一次掉落 + 一次拾取。
- 一次合成（二合、同类型唯一）。
- 一张卡的**装备态**（选一个简单触发绑定，如 onHit + slow）。
- 一张卡的**消耗态**（拖入战场落点释放，如 groundZone）。
- 一个触发器 + 一个状态效果（走 statusSystem 的控制预算/冲突仲裁）。
- 核心 GameEvent 传给 Unity 表现层（Unity 自行实现表现，不进 core）。
- 与 H5 黄金 fixture 对照（§六）。

通过后按此顺序扩张：弹道 → 控制 → 领域 → 防御 → 经济 → 神池/遗物/进化。

三、核心更新契约（H5 与 Unity 必须逐条一致）

3.1 单帧执行顺序（来自 `core/updateGame.ts`，权威）

每帧 `updateGame(state, config, rng, dt)`：

1. **门禁**：`mode !== 'playing'` 或 `paused` 或存在待处理决策（`decisions.current !== null` 或 `decisions.pending.length > 0`）→ 直接返回空事件，**不推进时间**。
2. **波间**：`intermission.active` 为真 → `state.time += dt` 后走 `tickBetween`，返回其事件（波间只推进波间逻辑，不跑战斗）。
3. 否则战斗帧，严格按此顺序：
 1. `state.time += dt`
 2. `tickOrdinaryDropBudget(dt)`（普通掉落额度累积）
 3. `updateTurret` → 事件（索敌 + 开火）
 4. `tickSpawns`（出怪）
 5. `updateBullets` → 事件
 6. `moveEnemies` → 事件
 7. `tickBountySystem` → 事件
 8. `tickEffects` → 事件（区域/光环/召唤物/护盾/状态/interval 绑定统一在实体推进之后 tick）
 9. `tickDrops` → 事件
 10. `updateParticles`（纯表现，Unity 可自行实现，不影响规则）
 11. `advanceWavePhase` → 事件（波次推进/结算）
4. 返回累积的 `GameEvent[]`。

不变量：暂停与"有待决策"时时间不推进；波间与战斗互斥；效果运行时始终在实体推进之后。任一顺序变化都可能造成两端结果分叉。

3.2 待 H5 侧补全并写入规格（Unity 需要，H5 T0.4 会正式产出）

- **RNG 播种规则**：`rng: () => number` 注入点、每系统内的调用次序（决定掉落/暴击/权重抽样的可重复性）。
- **伤害结算顺序**：`damageSystem.dealDamage` 内的减免/护盾/反伤/处决判定次序。
- **触发器重入/递归规则**：`split/chain` 的 `maxDepth`、`rider` 挂载、`cooldownSeconds` 语义。
- **状态冲突仲裁**：`statusSystem` 取最强/延时、控制预算 `controlBudgetDenies`。
- **派生属性对账**：装备变化后 `reconcileMaxHp / totalDamage` 等 `AFFIX_SINKS.globalConsumer` 的重算时机。
- **局内热更策略**：一局内是否允许换配置（当前：`variant` 切换 = 带参重载，保证一局内不漂移）。

3.3 数值语义

- 单位：像素 → 世界单位的换算比在规格书写死。
 - 浮点：**做统计一致，不做逐位一致**（见 § 六容差）。
 - 时间步：约定 `dtCap`（单帧 `dt` 上限）以防长卡顿放大积分误差；两端一致。
-

四、效果解释器移植要点

- 卡 = 数据 (JSON) + 通用解释器 (触发器 → 效果原子)。Unity 侧只写解释器，禁止为某张卡写 if —— 与 H5 架构规则一致。
 - 参数已是 "按 atom 判别的强类型" (EffectDef 判别联合，见 `core/effects/defs.ts`)。Unity 侧据此生成 C# DTO 或自定义反序列化；默认值一律以 `core/effects/atomContract.ts` 的 `ATOM_CONTRACT` 为准 (H5 运行时也从这张表读兜底，不再有内联默认值)，其中 `consumeDefault` / `passiveDefault` / `variantDefaults` 表示同一参数在不同结算路径下的兜底差异，Unity 必须一并实现。
 - 一期只需实现链路里用到的原子 (如 `slow` / `groundZone`)；扩张阶段按类别补齐。
-

五、配置消费规则 (D3)

- JSON 是唯一权威源。Unity 直接读 `config/base/*.json` 与 `variant` 覆盖 (深合并语义见 `config/loader.ts`)。
 - Unity 可把 JSON 导入为 `ScriptableObject` 作为运行时缓存/资产，但 SO 不是第二套真相，不得在 Unity 里手工改数值再回写。改数值一律回到 JSON。
 - 文案：Unity Localization 的 String Table 由同一份外部文案表 (`data/texts.json` 及后续本地化表) 导入。
 - 索敌抽象：一期只有自动索敌。请把索敌封装成接口 `ITargetProvider` (`AutoTargetProvider` 先行，`ManualAimTargetProvider` 预留)，以便将来加手动瞄准不改攻击系统。
-

六、跨引擎黄金对照协议

完整规格见 `docs/黄金回放_fixture规格.md` (字段定义、rng 算法与自检向量、逐项容差、排查顺序、如何新增 fixture)。要点：

- H5 侧已产出 5 个 fixture (`tests/golden/`)：`<id>.spec.json` 是输入，`<id>.summary.json` 是期望输出，两者都是 JSON，Unity 直接读同一批文件。
 - 覆盖：①纯战斗切片 (无决策无输入，Unity 一期即可完整复刻) ②消耗态释放 ③合成升星+装备态 ④三波通关 ⑤突破失败。
 - `rng = mulberry32`，算法与常量见规格书 § 2；Unity 端第一件事是跑通 `seed=42` 的前 8 抽自检向量。
 - 验收：关键语义必须一致 (事件类型序列、掉落序列、通关/失败结果、波次推进、`rng.draws` 抽取次数)；数值按规格书 § 5 的逐项容差 (H5 内部是逐位相等，容差只对 Unity 生效)。
 - 这是 "复刻正确" 的唯一客观验收标准；任何一致性失败先查 § 3.1 执行顺序与 § 3.2 判定次序 (规格书 § 6 给了排查顺序)。
-

七、给 Unity 开发者的三条硬约束

1. 先切片，后扩张：一期不追求功能齐全；端到端跑通 + 黄金对照通过，才逐类扩张。
2. core 不依赖引擎：Unity 的规则层同样只接收 `state + config + dt + rng` 返回状态变更与事件；表现/输入/UI 分层，副作用由事件驱动。

3. **配置只读**：一切数值来自 JSON；发现配置问题反馈给 H5 侧改 JSON，不在 Unity 私自改。

黄金回放 fixture 规格（T0.5）

日期：2026-07-26 | 交付对象：Unity 开发者 + H5 侧维护者 上游：docs/接下来任务计划_v1.md § T0.5、docs/Unity移植_纵向切片交付说明.md § 六（对照协议）

本文件定义**跨引擎一致性基线**：一组「固定 seed + 固定配置 + 脚本化输入」的对局录像，以及它们的结束态摘要。这是"复刻正确"的**唯一客观验收标准**——任何一致性失败，先查交付说明 § 3.1 的单帧执行顺序与 § 3.2 的判定次序。

0. 文件清单

路径	作用
src/core/rng.ts	可播种 rng (mulberry32) + 计数包装。 Unity 必须实现同一算法 ，见 § 2
src/core/replay/record.ts	录制/重放 harness：runReplay(spec) → summary
tests/golden/<id>.spec.json	输入 (ReplaySpec) ——Unity 直接读这份文件重放
tests/golden/<id>.summary.json	期望输出 (ReplaySummary) ——Unity 与这份比对
tests/goldenReplay.test.ts	H5 侧只读回放测试（逐位相等 + 连跑两次自治）
scripts/recordGoldenReplay.ts	唯一写盘入口：npm run replay:record [id...]

只读约定：回放测试永不写盘。fixture 只能由 npm run replay:record 重生成，且**重生成即意味着你承认行为变了**——提交时必须在 PR/commit 说明差异原因。

1. 五个 fixture 与覆盖面

id	场景	帧数	结果	覆盖	Unity 一期可复刻?
01-slice-combat	第 1 波纯战斗 16s	480	进行中	出怪 / 索敌 / 开火 / 伤害 / 击杀掉落 / 掉落过期	☑ 完全可复刻 （无决策、无输入）
02-slice-consumable	消耗态落点释放	900	进行中	拾取 / 二合 / 消耗释放 ×2 / freeze 状态	⚠ 需再实现拾取与消耗态
03-slice-merge-equip	合成升星 + 装备态	900	进行中	四合三 / 进化分支决策 / 装备 3★ / 词条掷点	⚠ 需再实现合成、装备、进化分支
04-run-victory	dev-short 三波通关	≤12000	胜利	完整链路：神池抽选 / 波间决策 / 波末 Boss / 遗物 / 结算	☑二期以后
05-run-defeat	低心防被突破	≤9000	失败	breakthrough 承伤 / 失败结算	⚠ 需实现突破伤害

建议接入顺序：Unity 一期只对 01 逐项比对；每扩张一个类别，再纳入下一个 fixture。每个 fixture 实际触发了哪些子系统，看其 summary 的 eventCounts ——那是权威清单，不要凭表格猜。

2. RNG: mulberry32（两端必须逐位一致）

```
state = seed >>> 0 // uint32
每次抽取：
state = (state + 0x6d2b79f5) mod 2^32
t = state
t = imul(t XOR (t >>> 15), t OR 1)
t = t XOR (t + imul(t XOR (t >>> 7), t OR 61))
return (t XOR (t >>> 14)) >>> 0 / 4294967296
```

- `imul` = 32 位有符号整数乘法（取低 32 位）；C# 用 `unchecked((int)a * (int)b)`。
 - `>>>` = 无符号右移；`>>> 0` = 转 `uint32`。
 - 全过程只有最后一步除以 `2^32` 是浮点，因此序列可逐位复刻。
 - **自检向量**（seed=42 的前 8 抽，保留 12 位小数）：0.60110375192, 0.448290558998, 0.85246579349, 0.669734041439, 0.174813898746, 0.526592542185, 0.27322799433, 0.624744653935 Unity 端第一件事就是跑通这 8 个数；对不上就不必往下比了。
 - **硬约束：**`src/core/**` 内禁止出现 `Math.random`（由 `tests/goldenReplay.test.ts` 守住）；rng 一律由调用方法注入。
-

3. ReplaySpec 字段

```
{
  "id": "01-slice-combat", // 主键，与文件名一致
  "description": ".....", // 人读说明，不参与比对
  "seed": 42, // rng 种子
  "variants": ["dev-short"], // 配置 variant 名单（深合并覆盖 base，语义见 config/loader.ts）
  "difficulty": "hell", // 可选，缺省 'hell'
  "dt": 0.03333333333333333, // 固定时间步（秒）= 1/30
  "frames": 480, // 最多推进帧数；对局提前结束则提前收尾
  "start": "wave1", // 'wave1' 直接开第 1 波（不产生开局决策）；'run' 走开局波间（含神池抽选）
  "decisionPolicy": "firstCandidate", // 待决策时的确定性选择：第一个 / 最后一个候选
  "overrides": { "hp": 60, "maxHp": 60, "damage": 6, "fireRate": 0, "range": 60, "dropChance": 0 },
  "inputs": [ /* 见 §4 */ ]
}
```

为什么需要 `decisionPolicy`：`updateGame` 在 `decisions.current !== null` 时直接返回、**不推进时间**（交付说明 §3.1 门禁）。没有选择策略，任何跨波对局都会卡死在决策上。策略是纯确定性的：取候选列表的首/末项。

执行顺序（每帧，Unity 必须照此实现）：

1. `updateGame(state, config, rng, dt)`

- 2. 若产生了待决策，就地按 `decisionPolicy` 依次清空决策队列（同一帧内，最多 16 次，防御死循环）
- 3. 执行本帧的脚本输入（同帧多条按声明顺序）
- 4. 记录新出现的掉落、累计伤害
- 5. `mode !== 'playing'` 则收尾

4. 脚本化输入（ReplayInput）

每条输入带 `frame`，并可选 `repeatEvery` / `repeatUntil` 做周期展开（`repeatUntil` 缺省 = `frames`）。

kind	字段	语义
<code>spawnDrop</code>	<code>x, y, cardType, star</code>	场景搭建用 ：直接生成一枚掉落，避免依赖掉落 <code>rng</code> 才能构造合成场景。这是唯一不对应玩家动作的输入
<code>collectAt</code>	<code>x, y, radius?</code>	点击拾取 <code>(x,y)</code> 半径内最近的掉落（ <code>radius</code> 缺省 = <code>economy.drops.pickupRadius</code> ）
<code>collectFirstDrop</code>	—	点击最早出现的地面掉落；场上无掉落时无动作
<code>moveOrSwap</code>	<code>source, index, targetKind, targetIndex</code>	拖拽：手牌/装备栏之间移动或交换
<code>consumeAt</code>	<code>index, x, y</code>	消耗释放：手牌 <code>index</code> 的卡在 <code>(x,y)</code> 落点释放
<code>confirmIntermission</code>	—	波间「准备完毕」，跳过剩余自由整备时间

空动作是合法的：目标槽位为空、场上无掉落等情况下动作静默无效——这保证了输入脚本不必预知运行时状态，重放仍然确定。

5. ReplaySummary 字段与比对口径

字段	类型	Unity 比对口径
<code>spec</code>	输入回写	必须逐字相等 （确认两端跑的是同一份输入）
<code>framesRun / mode / win</code>	标量	必须相等 （通关/失败结论不允许有分歧）
<code>wave</code>	<code>{wave, phase, spawnLeft, waveSpawnQuota, intermission}</code>	必须相等 （波次推进是关键语义）
<code>eventSequence</code>	<code>[[{frame, type}]]</code>	类型序列必须相等 ；帧号允许 ± 1 帧偏移（浮点累积）
<code>eventCounts</code>	<code>Record<type, number></code>	必须相等
<code>dropSequence</code>	<code>[[{frame, action, dropId, kind, cardType, star}]]</code>	<code>action / cardType / star</code> 序列必须相等； <code>frame</code> 同上允许 ± 1
<code>cards / equipment</code>		必须相等

字段	类型	Unity 比对口径
	槽位快照（含 <code>affixes</code> 、 <code>evolutionPath</code> ）	
<code>wildcards / relics / counters</code>	计数与 id 列表	必须相等
<code>enemiesRemaining</code>	整数	必须相等
<code>hp / maxHp</code>	浮点	相对误差 $\leq 1e-6$
<code>cumulativeDamageDealt / cumulativeDamageTaken</code>	浮点	相对误差 $\leq 1e-4$ （逐帧累加，误差会放大）
<code>rng.draws</code>	整数	必须相等——这是 rng 调用次序的指纹，比任何数值都更早暴露分叉
<code>rng.last</code>	浮点	必须相等（同一序列的同一位置）

H5 内部是逐位相等（`toEqual` 全字段严格比对），上述容差只对 Unity 生效。

两个易错定义，Unity 必须照抄：

- `cumulativeDamageDealt` = 逐帧对**同 id** 敌人取 `max(0, 前帧hp - 本帧hp)` 求和。敌人消失那一帧的剩余血量**不计入**（死亡由 `counters.kills` 表达）。
- `cumulativeDamageTaken` = `breakthrough` 与 `bossContactDamage` 事件携带的 `damage` 之和。

6. 排查顺序（对不上时）

1. `rng.draws` 对不上 → 某个系统多抽/少抽了一次随机数：核对交付说明 § 3.1 的单帧执行顺序与各系统内部的抽取次序。
2. `rng.draws` 一致但 `eventSequence` 分叉 → 判定次序问题（伤害结算/状态仲裁/触发器重入）：核对 § 3.2。
3. 事件一致但数值超容差 → 积分/取整差异：核对 `dt` 语义、`dtCap`、像素→世界单位换算。
4. 只有 `cards / equipment` 不一致 → 合成/装备/词条掷点的 `rng` 消费位置不同。

7. 如何新增一个 fixture

1. 在 `tests/golden/` 新建 `<NN>-<名字>.spec.json`（编号决定录制与断言顺序）。
2. `npm run replay:record <id>` 生成 `<id>.summary.json`；不带参数则全量重录。
3. 在 `tests/goldenReplay.test.ts` 的 fixture 清单里登记 id，并按需补一条语义断言（"这个 fixture 到底覆盖了什么"）。
4. `npm run test` 全绿后连同 summary 一起提交。

改动 fixture 的纪律：summary 变了就是行为变了。先确认这是有意的玩法/规则变更，再重录；不要为了让测试变绿而重录。